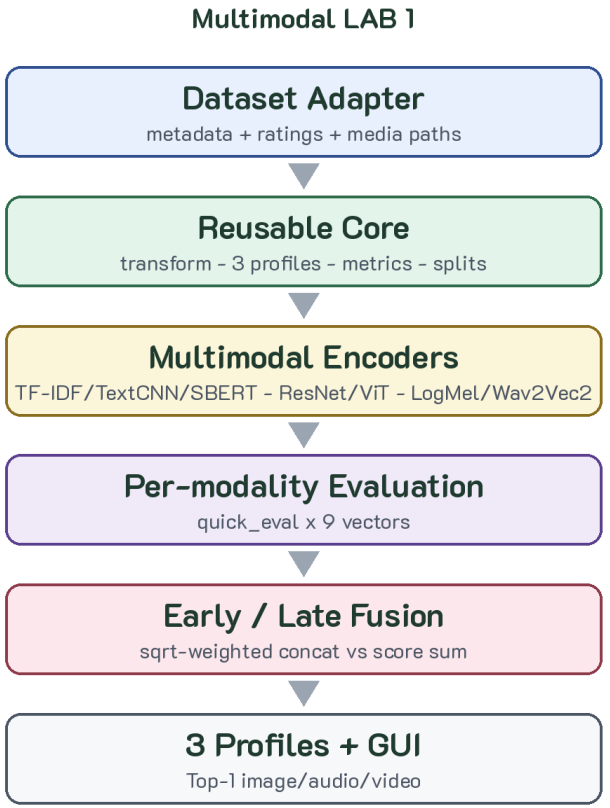
LAB 4:
Multimodal Foundations

พื้นฐานระบบแนะนำหลายสื่อ

204729
Recommender Systems

ผู้ช่วยศาสตราจารย์ ดร.ดุษฎี ประเสริฐธิติพงษ์

Multimodal Foundations
Vector - Fusion - GUI



อธิบายแนวคิดของ Source Code ทีละ Part

CNN / Transformer encoders - Fusion - Chapter 2 metrics - Gradio

COMPUTER SCIENCE

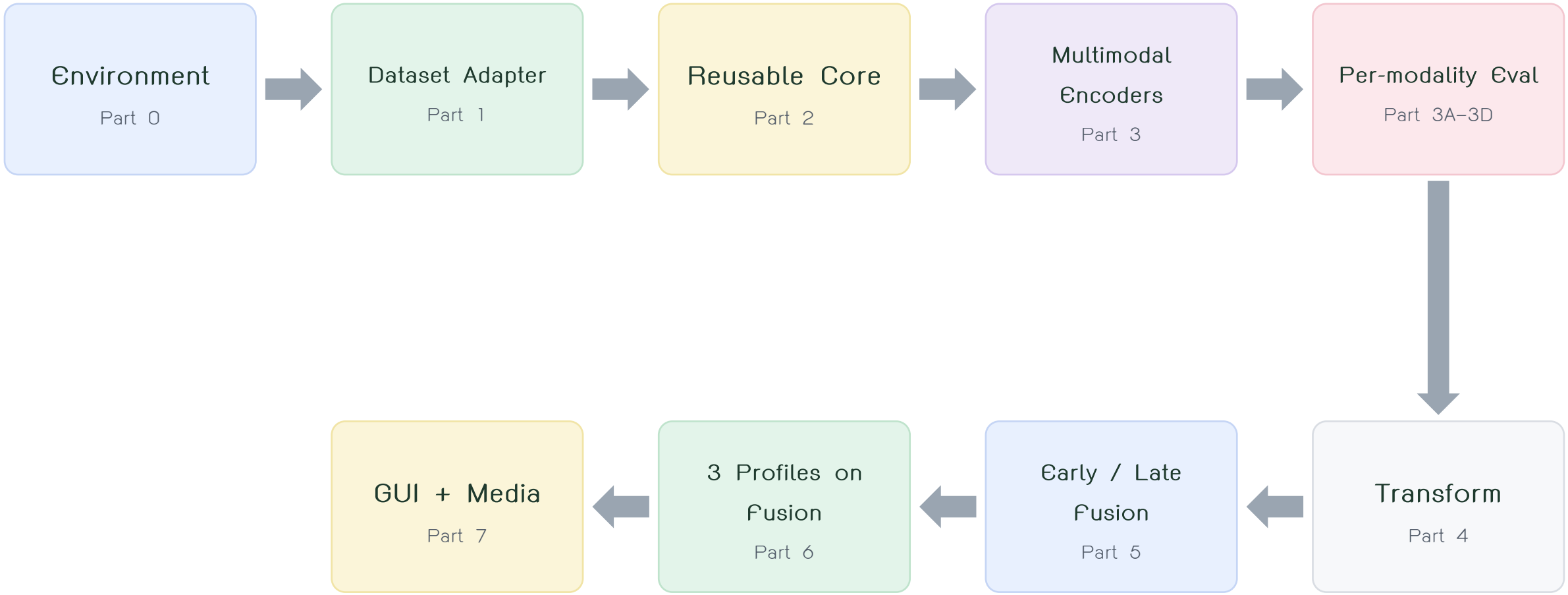
Multimodal LAB 1 คืออะไร

ภาพรวม

- Notebook ต้นแบบที่แยก **Dataset Adapter (Part 1)** ออกจาก **Reusable Core (Part 2)** อย่างชัดเจน → เปลี่ยน Dataset ได้โดยแก้ Part 1 เท่านั้น
- **Part 3 Multimodal Encoders** ตามเอกสารบทที่ 3–4: Text (TF-IDF / TextCNN / SBERT), Image (ResNet18 / ViT-B/16), Audio (Log-Mel → ResNet18 / Wav2Vec2), Video (frame sampling → CNN/ViT → temporal mean pooling) ทุกตัวจบด้วย **pooling + L2 normalization** และ cache เป็น `.npy`
- ประเมิน **รายสื่อ** ด้วย `quick_eval` (3 profile × LOO) แล้วดูผลของ **Transformation 4 แบบ** บน SBERT
- **Part 5 Fusion:** Early (ต่อเวกเตอร์ถ่วงน้ำหนัก √w) vs Late (รวมคะแนน cosine รายสื่อ) ประเมินภายใต้ protocol เดียวกัน ; **Part 6** เทียบ 3 User Profile บน fused space
- **Part 7 GUI (gr.Blocks):** History, Profile preview, Top-K พร้อม **image / audio / video ของอันดับ 1**
- Core Functions (profile / rank / metric / split / evaluate) เป็นชุดเดียวกับ Text LAB 1–3 เกือบทั้งหมด ต่างที่ชื่อฟังก์ชันและเพิ่ม `PCA+L2`, `label`, `quick_eval`

แนวคิดหลัก: ทุกสื่อ → fixed-length vector → L2 → User Profile → cosine → Top-K → ประเมินตามบทที่ 2

โครงสร้าง Notebook (Part 0 → Part 7)



Part 1–3 คือ “การสร้างเวกเตอร์” ; Part 4–7 คือ “การใช้เวกเตอร์”

แผนที่ของ Source Code Multimodal LAB 1

Part	หน้าที่	ฟังก์ชัน / ตัวแปรสำคัญ	ผลลัพธ์
0	ติดตั้ง + import	`sentence-transformers`, `transformers`, `librosa`, `gradio`	environment
1	Dataset Adapter	`ITEM_ALIASES`, `RATING_ALIASES`, `MEDIA_DIRS`, `_find_media`, `load_dataset`	`items` (+`text_content`, `*_path`), `ratings`, `ITEM_INDEX`
2	Reusable Core	`transform_vectors` (5 โหมด), `profile_*` ×3, `make_profile`, `rank_by_vector`, metric ×13, `split_*` ×3, `evaluate_vector_space`, `quick_eval`	เครื่องมือกลาง
3	Multimodal Encoders	`cache_vec`, `build_tfidf`, `build_textcnn_vectors`, `RESNET/VIT`, `encode_pil`, `audio_to_image`, `_audio_w2v`, `frames`, `_video`, `VECTOR_LIBRARY`	9 เมทริกซ์
3A–3D	ประเมินรายสื่อ	`quick_eval(X, label)`	ตาราง profile × metric ต่อเวกเตอร์
4	Transformation บน SBERT	`transform_vectors(X_text_sbert, mode)`	4 แถว
5	Fusion	`early_fusion`, `DEFAULT_MODALITIES`, `X_fused`, `late_fusion_rank`, `evaluate_late_fusion`	Early vs Late
6	3 Profiles บน fused	`evaluate_vector_space(X_fused, ..., p)`	3 แถว
7	GUI	`FUSED_SPACES`, `gui`, `gr.Blocks`	History + Profile + Top-K + media

1

Dataset Adapter

Part 0 – Part 1: จุดเดียวที่ขึ้นกับ Dataset — alias ของคอลัมน์ โฟลเดอร์สื่อ และนามสกุลไฟล์

- `load_dataset()` คืน `items`, `ratings` ใน canonical schema พร้อม `text_content` และ `image_path / audio_path / video_path`

Part 1: ค่าคงที่ของ Adapter (แก้เฉพาะส่วนนี้เมื่อเปลี่ยน Dataset) (1/2)

```
BASE = Path("/content/drive/MyDrive/Colab Notebooks")
DATA_ROOT = BASE / "Dataset_Chapter3_4"
DATA_DIR = DATA_ROOT / "data" if (DATA_ROOT / "data").exists() else
        DATA_ROOT

ITEM_FILE = DATA_DIR / "metadata.csv"
RATING_FILE = DATA_DIR / "ratings.csv"

ITEM_ALIASES = {"movie_id": "item_id", "id": "item_id", "movie_title":
        "title", "name": "title",
                "genres": "genre", "description": "synopsis", "plot":
                        "synopsis", "summary": "synopsis"}
RATING_ALIASES = {"uid": "user_id", "user": "user_id", "iid":
        "item_id", "movie_id": "item_id", "score": "rating"}

MEDIA_DIRS = {"image": DATA_DIR / "images", "audio": DATA_DIR /
        "audio", "video": DATA_DIR / "video"}
```

อธิบายโค้ด

ทุกอย่างที่ “ขึ้นกับ Dataset” อยู่ในค่าคงที่

- ชื่อไฟล์, alias ของคอลัมน์, โฟลเดอร์สื่อ และนามสกุลที่รองรับ ถูกประกาศเป็น **ค่าคงที่ตัวพิมพ์ใหญ่** ⟶ เปลี่ยน Dataset = แก้ค่าเหล่านี้ ไม่ต้องแตะ Core
- `_rename_if_needed` เปลี่ยนชื่อเฉพาะเมื่อยังไม่มีชื่อมาตรฐาน ; `_find_media` ลองทุกนามสกุลแล้วคืน `None` ถ้าไม่พบ (ไม่ใช่ path ปลอม) ⟶ encoder ใน Part 3 ตรวจ `if p` ได้
- `DATA_DIR` รองรับทั้งกรณีมี/ไม่มีโฟลเดอร์ย่อย `data/`
- ข้อตกลง: ชื่อไฟล์สื่อ = `item_id` + นามสกุล

Part 1: ค่าคงที่ของ Adapter (แก้เฉพาะส่วนนี้เมื่อเปลี่ยน Dataset) (2/2)

```python
MEDIA_EXTS = {"image": [".png", ".jpg", ".jpeg", ".webp"],
              "audio": [".wav", ".mp3", ".flac", ".m4a"],
              "video": [".mp4", ".mov", ".avi", ".mkv"]}

def _rename_if_needed(df, aliases):
    df = df.copy()
    for old, new in aliases.items():
        if old in df.columns and new not in df.columns:
            df = df.rename(columns={old: new})
    return df

def _find_media(folder, item_id, exts):
    for ext in exts:
        p = folder / f"{item_id}{ext}"
        if p.exists():
            return str(p)
    return None
```

อธิบายโค้ด

ทุกอย่างที่ “ขึ้นกับ Dataset” อยู่ในค่าคงที่

- ชื่อไฟล์, alias ของคอลัมน์, โฟลเดอร์สื่อ และนามสกุลที่รองรับ ถูกประกาศเป็น **ค่าคงที่ตัวพิมพ์ใหญ่** ⟶ เปลี่ยน Dataset = แก้ค่าเหล่านี้ ไม่ต้องแตะ Core
- `_rename_if_needed` เปลี่ยนชื่อเฉพาะเมื่อยังไม่มีชื่อมาตรฐาน ; `_find_media` ลองทุกนามสกุลแล้วคืน `None` ถ้าไม่พบ (ไม่ใช่ path ปลอม) ⟶ encoder ใน Part 3 ตรวจ `if p` ได้
- `DATA_DIR` รองรับทั้งกรณีมี/ไม่มีโฟลเดอร์ย่อย `data/`
- ข้อตกลง: ชื่อไฟล์สื่อ = `item_id` + นามสกุล

Part 1: load_dataset และตัวแปร global

```python
def load_dataset():
    items = _rename_if_needed(pd.read_csv(ITEM_FILE), ITEM_ALIASES)
    ratings = _rename_if_needed(pd.read_csv(RATING_FILE),
            RATING_ALIASES)
    need_i = ["item_id", "title", "genre", "synopsis"]
    need_r = ["user_id", "item_id", "rating"]
    miss_i = [c for c in need_i if c not in items.columns]
    miss_r = [c for c in need_r if c not in ratings.columns]
    if miss_i or miss_r:
        raise ValueError(f"Missing item columns={miss_i}, rating
                columns={miss_r}")
    items["item_id"] = items["item_id"].astype(str)
    ratings["item_id"] = ratings["item_id"].astype(str)
    ratings["user_id"] = ratings["user_id"].astype(str)
    ratings["rating"] = pd.to_numeric(ratings["rating"])
    for c in ["title", "genre", "synopsis"]:
        items[c] = items[c].fillna("").astype(str)
    items["text_content"] = (items["genre"] + " " +
            items["synopsis"]).str.strip()
    for modality in ["image", "audio", "video"]:
        items[f"{modality}_path"] = items["item_id"].apply(
            lambda x: _find_media(MEDIA_DIRS[modality], x,
                    MEDIA_EXTS[modality]))
    return items, ratings

items, ratings = load_dataset()
ITEM_INDEX = {iid: i for i, iid in enumerate(items["item_id"])}
```

อธิบายโค้ด

canonical schema สำหรับ 4 สื่อ

- `items`: `item_id, title, genre, synopsis, text_content, image_path, audio_path, video_path` ; `ratings`: `user_id, item_id, rating` — ทุก id เป็น `str`
- ตรวจคอลัมน์บังคับแล้ว `raise ValueError` ระบุทั้งสองตาราง
- `text_content` = genre + synopsis เป็น input ของ encoder ข้อความทั้ง 3 แบบ
- `*_path` เป็น `None` เมื่อไม่มีไฟล์ ⟶ encoder แทนด้วยภาพดำ/เวกเตอร์ศูนย์ แทนที่จะ crash
- `ITEM_INDEX` map item_id ⟶ แถวของทุกเมทริกซ์ใน `VECTOR_LIBRARY`

2

Reusable Core

Part 2: ไม่ผูกกับ MovieMini-12 — ขอเพียง items / ratings อยู่ใน canonical schema

- `transform_vectors` 5 โหมด (เพิ่ม Raw และ PCA+L2)
- 3 User Profile ผ่าน `make_profile` ; `rank_by_vector` ; metric 13 ตัว ; split 3 แบบ ; `evaluate_vector_space` ; `quick_eval`

Part 2: transform_vectors (5 โหมด) และ _unit

```python
LIKED_THRESHOLD = 4.0
DISLIKED_THRESHOLD = 2.0
N_USERS = max(1, ratings["user_id"].nunique())
POPULARITY = ratings.groupby("item_id")["user_id"].nunique().to_dict()

def transform_vectors(X, mode="L2", pca_dim=16):
    X = np.asarray(X, dtype=float)
    if mode == "Raw":
        return X.copy()
    if mode == "L2":
        return normalize(X)
    if mode == "Center+L2":
        return normalize(X - X.mean(axis=0, keepdims=True))
    if mode == "Standard+L2":
        return normalize(StandardScaler().fit_transform(X))
    if mode == "PCA+L2":
        d = min(pca_dim, max(1, X.shape[0] - 1), X.shape[1])
        return normalize(PCA(n_components=d,
                 random_state=42).fit_transform(X))
    raise ValueError(mode)

def _unit(v):
    v = np.asarray(v, dtype=float)
    n = np.linalg.norm(v)
    return v if n == 0 else v / n
```

อธิบายโค้ด

geometry ของ embedding space

- Raw (ไม่ทำอะไร) และ L2 (ทิศทางล้วน) ; encoder ส่วนใหญ่ใน Part 3 L2 มาแล้ว ⟶ L2 แทบไม่เปลี่ยน
- Center+L2: ลบค่าเฉลี่ยของ catalog — สำคัญกับ embedding จาก deep model ที่มี “ทิศทางร่วม” (anisotropy) ทำให้ cosine ทุกคู่สูง
- Standard+L2: ทุกมิติ mean 0 / std 1 — ระวัง noise กับ 12 item
- PCA+L2 (ใหม่): ลดมิติเหลือ ≤ 16 และ ≤ n−1 = 11 ⟶ ตัด noise ของ embedding 512/768 มิติจากตัวอย่างเพียง 12 ; random_state=42
- POPULARITY นับผู้ใช้ทุกคนที่ให้คะแนน ⟶ ใช้ใน Novelty ; _unit = L2 ทนเวกเตอร์ศูนย์

Part 2: 3 User Profiles และ make_profile (1/2)

```python
def profile_positive_only(uid, X, rdf):
    r = rdf[rdf["user_id"].astype(str) == str(uid)]
    liked = r[r["rating"] >= LIKED_THRESHOLD]
    idx = [ITEM_INDEX[str(i)] for i in liked["item_id"] if str(i) in
           ITEM_INDEX]
    if not idx:
        idx = [ITEM_INDEX[str(i)] for i in r["item_id"] if str(i) in
               ITEM_INDEX]
    return _unit(X[idx].mean(axis=0)) if idx else np.zeros(X.shape[1])

def profile_rating_centered(uid, X, rdf):
    r = rdf[rdf["user_id"].astype(str) == str(uid)].copy()
    r = r[r["item_id"].astype(str).isin(ITEM_INDEX)]
    if r.empty: return np.zeros(X.shape[1])
    idx = np.array([ITEM_INDEX[str(i)] for i in r["item_id"]])
    w = r["rating"].astype(float).values - float(r["rating"].mean())
           # rating - user mean
    den = np.abs(w).sum()
    p = X[idx].mean(axis=0) if den <= 1e-12 else (X[idx] * w[:,
           None]).sum(axis=0) / den
    return _unit(p)

def profile_rocchio(uid, X, rdf, alpha=1.0, beta=1.0):
    r = rdf[rdf["user_id"].astype(str) == str(uid)]
    pos = r[r["rating"] >= LIKED_THRESHOLD]; neg = r[r["rating"] <=
```

อธิบายโค้ด

สามวิธีใช้ rating สร้างโปรไฟล์

- Positive-only = mean ของเวกเตอร์หนังที่ rating ≥ 4 (fallback: ทุกเรื่องที่เคยดู)
- Rating-centered = ถ่วงน้ำหนักด้วย **rating − ค่าเฉลี่ยของผู้ใช้เอง** หาร $\sum|w|$ ⟶ แก้ user bias ; ผู้ใช้ให้คะแนนเท่ากันหมด ⟶ fallback mean
- Rocchio = α·mean(liked) − β·mean(disliked ≤ 2) ; rating 3 ไม่ถูกใช้ ; α, β ปรับได้จาก GUI
- ทุกสูตรทำงานกับ `X` ใด ๆ (TF-IDF, ResNet, Wav2Vec2, fused) เพราะเป็นเพียงการเฉลี่ยแถวของเมทริกซ์ ⟶ นี่คือเหตุผลที่ Core “ไม่ผูกกับสื่อ”
- `make_profile` เป็นทางเข้าเดียว (`PROFILE_METHODS` เป็น list ไม่ใช่ dict ต่างจาก Text LAB) ; ปิดท้ายด้วย `_unit` ทุกสูตร

Part 2: 3 User Profiles และ make_profile (2/2)

```
            DISLIKED_THRESHOLD]
    pi = [ITEM_INDEX[str(i)] for i in pos["item_id"] if str(i) in
            ITEM_INDEX]
    ni = [ITEM_INDEX[str(i)] for i in neg["item_id"] if str(i) in
            ITEM_INDEX]
    vp = X[pi].mean(axis=0) if pi else np.zeros(X.shape[1])
    vn = X[ni].mean(axis=0) if ni else np.zeros(X.shape[1])
    p = alpha * vp - beta * vn
    if np.linalg.norm(p) == 0:
        hi = [ITEM_INDEX[str(i)] for i in r["item_id"] if str(i) in
                ITEM_INDEX]
        p = X[hi].mean(axis=0) if hi else p
    return _unit(p)

PROFILE_METHODS = ["Positive-only", "Rating-centered", "Rocchio"]

def make_profile(uid, X, rdf, method="Rating-centered", alpha=1.0,
        beta=1.0):
    if method == "Positive-only": return profile_positive_only(uid, X,
            rdf)
    if method == "Rating-centered": return profile_rating_centered(uid,
            X, rdf)
    if method == "Rocchio": return profile_rocchio(uid, X, rdf, alpha,
            beta)
    raise ValueError(method)
```

อธิบายโค้ด

สามวิธีใช้ rating สร้างโปรไฟล์

- Positive-only = mean ของเวกเตอร์หนังที่ rating ≥ 4 (fallback: ทุกเรื่องที่เคยดู)
- Rating-centered = ถ่วงน้ำหนักด้วย **rating − ค่าเฉลี่ยของผู้ใช้เอง** หาร $\Sigma|w|$ ⟶ แก้ user bias ; ผู้ใช้ให้คะแนนเท่ากันหมด ⟶ fallback mean
- Rocchio = α·mean(liked) − β·mean(disliked ≤ 2) ; rating 3 ไม่ถูกใช้ ; α, β ปรับได้จาก GUI
- ทุกสูตรทำงานกับ `X` ใด ๆ (TF-IDF, ResNet, Wav2Vec2, fused) เพราะเป็นเพียงการเฉลี่ยแถวของเมทริกซ์ ⟶ นี่คือเหตุผลที่ Core “ไม่ผูกกับสื่อ”
- **make_profile** เป็นทางเข้าเดียว (**PROFILE_METHODS** เป็น list ไม่ใช่ dict ต่างจาก Text LAB) ; ปิดท้ายด้วย **_unit** ทุกสูตร

Part 2: rank_by_vector และ metric 13 ตัว (สรุป) (1/2)

```python
def rank_by_vector(uid, X, rdf, method="Rating-centered",
        candidates=None, alpha=1.0, beta=1.0):
    p = make_profile(uid, X, rdf, method, alpha, beta)
    s = cosine_similarity(p.reshape(1, -1), X).ravel()
    seen = set(rdf.loc[rdf["user_id"].astype(str) == str(uid),
            "item_id"].astype(str))
    cand = candidates if candidates is not None else [iid for iid in
            items["item_id"] if iid not in seen]
    ranked = sorted(cand, key=lambda iid: s[ITEM_INDEX[iid]],
            reverse=True)
    return ranked, s, p

# Accuracy / Ranking: precision_at_k, recall_at_k, f1_at_k, hr_at_k,
#                     reciprocal_rank, ap_at_k, ndcg_at_k
        (สูตรเดียวกับ Text LAB 1)
# Beyond-Accuracy:
def ild(recs, X):                          # mean(1 - cosine) ของคู่ใน
```

อธิบายโค้ด

จัดอันดับและวัดผล

- `rank_by_vector` คืน `(ranked, s, p)`: ลำดับ candidate, คะแนน cosine ของทุก item, และโปรไฟล์ — GUI ใช้ `p` แสดง preview, evaluate ใช้ `s` คำนวณ AUC
- `candidates=None` ⟶ ทุกเรื่องที่ไม่อยู่ใน `rdf` ; evaluate ส่ง candidate ที่เตรียมเอง (Full / Sampled)
- metric ทั้ง 13 ตัวเหมือน Text LAB 1 (ดูสไลด์ชุดนั้นสำหรับสูตรเต็ม) — ILD รับ `X` จึงวัดใน space ของเวกเตอร์ที่ประเมิน; ข้าม modality เทียบไม่ได้ตรง
- Calibration = 1 − total variation distance ระหว่างการกระจายแนวหนังของประวัติกับ Top-K

Part 2: rank_by_vector และ metric 13 ตัว (สรุป) (2/2)

```
        Top-K, วัดใน space X
    idx = [ITEM_INDEX[i] for i in recs if i in ITEM_INDEX]
    if len(idx) < 2: return 0.0
    D = 1 - cosine_similarity(X[idx]); v = D[np.triu_indices_from(D,
            k=1)]
    return float(v.mean()) if len(v) else 0.0
def novelty(recs):                          # -log2(popularity / N_USERS)
    vals = [-math.log2(max(POPULARITY.get(i, 1) / N_USERS, 1e-12)) for
            i in recs]
    return float(np.mean(vals)) if vals else 0.0
def calibration_genre(uid, recs, rdf):  # 1 - 0.5 * sum|p_hist - p_rec|
        (genre distribution)
    ...
def coverage(all_recs): return len(set(all_recs)) / len(items) if
        len(items) else 0.0
def gini(all_recs): ...                     # Gini ของความถี่ที่แต่ละ item
        ถูกแนะนำ
```

อธิบายโค้ด

จัดอันดับและวัดผล

- `rank_by_vector` คืน `(ranked, s, p)`: ลำดับ candidate, คะแนน cosine ของทุก item, และโปรไฟล์ — GUI ใช้ `p` แสดง preview, evaluate ใช้ `s` คำนวณ AUC
- `candidates=None` ⟶ ทุกเรื่องที่ไม่อยู่ใน `rdf` ; evaluate ส่ง candidate ที่เตรียมเอง (Full / Sampled)
- metric ทั้ง 13 ตัวเหมือน Text LAB 1 (ดูสไลด์ชุดนั้นสำหรับสูตรเต็ม) — ILD รับ `X` จึงวัดใน space ของเวกเตอร์ที่ประเมิน; ข้าม modality เทียบไม่ได้ตรง
- Calibration = 1 − total variation distance ระหว่างการกระจายแนวหนังของประวัติกับ Top-K

Part 2: splits, evaluate_vector_space และ quick_eval (1/3)

```
def split_loo(rdf): ...                  # held-out = positive ที่
        rating สูงสุด
def split_random_holdout(rdf, seed=42): ...   # held-out = positive สุ่ม
def split_temporal(rdf): ...             # ต้องมี timestamp มิฉะนั้นคืน
        None

def evaluate_vector_space(X, label="",
        profile_method="Rating-centered", k=3, folds=None,
                        candidate_mode="Full Catalog", n_neg=20,
                                alpha=1.0, beta=1.0, seed=42):
    folds = folds or split_loo(ratings)
    rng = np.random.default_rng(seed)
    rows = []; all_recs = []
    for uid, train, heldout in folds:
        seen = set(train.loc[train["user_id"].astype(str) == uid,
                "item_id"].astype(str))
        catalog = [i for i in items["item_id"] if i not in seen]
        if heldout not in catalog: catalog.append(heldout)
        if candidate_mode == "Sampled Negatives":
            neg = [i for i in catalog if i != heldout]
```

อธิบายโค้ด

engine เดียวสำหรับทุกเวกเตอร์

- `evaluate_vector_space` = `evaluate_config` ของ Text LAB + พารามิเตอร์ `label` ที่ถูกใส่ใน summary ⟶ ตารางที่รวมหลายเวกเตอร์อ่านได้ว่าแถวไหนมาจากอะไร
- held-out ถูก drop จาก `train` ก่อนสร้างโปรไฟล์และถูกบังคับให้อยู่ใน candidate ; Sampled Negatives ใช้ `rng` seed คงที่
- `quick_eval(X, label)` = วน 3 profile บน LOO + Full Catalog แล้วเลือก 8 คอลัมน์สำคัญ ⟶ ใช้เป็น “มาตรวัดมาตรฐาน” ของทุกเวกเตอร์ใน Part 3A–3D
- ทุกเวกเตอร์ในเล่มนี้ถูกวัดด้วยฟังก์ชันนี้ตัวเดียว ⟶ ผลต่างระหว่างตารางมาจาก encoder เท่านั้น

Part 2: splits, evaluate_vector_space และ quick_eval (2/3)

```
        take = min(n_neg, len(neg))
        cand = [heldout] + (list(rng.choice(neg, take,
                replace=False)) if take else [])
    else:
        cand = catalog
    ranked, s, _ = rank_by_vector(uid, X, train, profile_method,
            cand, alpha, beta)
    top = ranked[:k]; all_recs += top
    y = np.array([1 if i == heldout else 0 for i in ranked])
    ys = np.array([s[ITEM_INDEX[i]] for i in ranked])
    auc = roc_auc_score(y, ys) if len(np.unique(y)) == 2 else np.nan
    rows.append({"user_id": uid, "heldout": heldout,
                 f"Precision@{k}": precision_at_k(top, [heldout],
                        k), f"Recall@{k}": recall_at_k(top,
                                [heldout], k),
                 f"F1@{k}": f1_at_k(top, [heldout], k), f"HR@{k}":
                        hr_at_k(top, [heldout], k),
                 "MRR": reciprocal_rank(ranked, [heldout]),
                        f"MAP@{k}": ap_at_k(top, [heldout], k),
                 f"nDCG@{k}": ndcg_at_k(top, [heldout], k), "AUC":
```

อธิบายโค้ด

engine เดียวสำหรับทุกเวกเตอร์

- `evaluate_vector_space` = `evaluate_config` ของ Text LAB + พารามิเตอร์ `label` ที่ถูกใส่ใน summary ⟶ ตารางที่รวมหลายเวกเตอร์อ่านได้ว่าแถวไหนมาจากอะไร
- held-out ถูก drop จาก `train` ก่อนสร้างโปรไฟล์และถูกบังคับให้อยู่ใน candidate ; Sampled Negatives ใช้ `rng` seed คงที่
- `quick_eval(X, label)` = วน 3 profile บน LOO + Full Catalog แล้วเลือก 8 คอลัมน์สำคัญ ⟶ ใช้เป็น “มาตรวัดมาตรฐาน” ของทุกเวกเตอร์ใน Part 3A–3D
- ทุกเวกเตอร์ในเล่มนี้ถูกวัดด้วยฟังก์ชันนี้ตัวเดียว ⟶ ผลต่างระหว่างตารางมาจาก encoder เท่านั้น

Part 2: splits, evaluate_vector_space และ quick_eval (3/3)

```
                            auc,
                    "ILD": ild(top, X), "Novelty": novelty(top),
                            "Calibration": calibration_genre(uid, top,
                                    train)})
    d = pd.DataFrame(rows)
    s = d.select_dtypes(include=np.number).mean().to_dict() if len(d)
            else {}
    s["Coverage"] = coverage(all_recs); s["Gini"] = gini(all_recs);
            s["Label"] = label
    return pd.Series(s), d

def quick_eval(X, label):
    rows = []
    for p in PROFILE_METHODS:
        s, _ = evaluate_vector_space(X, label, p, k=3)
        rows.append({"Vector": label, "Profile": p, **s.to_dict()})
    cols = ["Vector", "Profile", "HR@3", "MRR", "nDCG@3", "Coverage",
            "Gini", "ILD", "Novelty", "Calibration"]
    return pd.DataFrame(rows)[[c for c in cols if c in
            pd.DataFrame(rows).columns]]
```

อธิบายโค้ด

engine เดียวสำหรับทุกเวกเตอร์

- `evaluate_vector_space` = `evaluate_config` ของ Text LAB + พารามิเตอร์ `label` ที่ถูกใส่ใน summary ⟶ ตารางที่รวมหลายเวกเตอร์อ่านได้ว่าแถวไหนมาจากอะไร
- held-out ถูก drop จาก `train` ก่อนสร้างโปรไฟล์และถูกบังคับให้อยู่ใน candidate ; Sampled Negatives ใช้ `rng` seed คงที่
- `quick_eval(X, label)` = วน 3 profile บน LOO + Full Catalog แล้วเลือก 8 คอลัมน์สำคัญ ⟶ ใช้เป็น “มาตรวัดมาตรฐาน” ของทุกเวกเตอร์ใน Part 3A–3D
- ทุกเวกเตอร์ในเล่มนี้ถูกวัดด้วยฟังก์ชันนี้ตัวเดียว ⟶ ผลต่างระหว่างตารางมาจาก encoder เท่านั้น

3

Multimodal Encoders

Part 3: Text / Image / Audio / Video ⟶ fixed-length vector ด้วย CNN และ Transformer

- `cache_vec(name, builder)` เก็บผลเป็น `.npy` ใน `vector_cache/` — รันซ้ำไม่ต้องคำนวณใหม่
- 9 เวกเตอร์ใน `VECTOR_LIBRARY`

Part 3: setup, cache_vec, TF-IDF และ SBERT (1/2)

```python
import torch, torch.nn as nn
from PIL import Image
import cv2, librosa
from torchvision.models import resnet18, ResNet18_Weights, vit_b_16,
        ViT_B_16_Weights
from sentence_transformers import SentenceTransformer
from transformers import AutoProcessor, AutoModel

DEVICE = torch.device("cuda" if torch.cuda.is_available() else "cpu")
CACHE = DATA_ROOT / "vector_cache"
CACHE.mkdir(parents=True, exist_ok=True)

def cache_vec(name, builder, rebuild=False):
    p = CACHE / f"{name}.npy"
    if p.exists() and not rebuild:
        print("cache:", name); return np.load(p)
```

อธิบายโค้ด

โครงที่ทุก encoder ใช้ร่วมกัน

- `cache_vec` รับ builder (ฟังก์ชันไม่มี argument) ⟶ ถ้ามีไฟล์ `.npy` แล้วโหลดทันที ไม่งั้นคำนวณแล้วบันทึกลง Drive ; `rebuild=True` บังคับคำนวณใหม่
- `DEVICE` เลือก GPU อัตโนมัติ ; โมเดล pretrained ทุกตัว `.eval()` + `torch.no_grad()` = ใช้เป็น feature extractor ไม่ฝึกต่อ
- TF-IDF ไม่ cache (เร็วและต้องเก็บ `TFIDF_VECTORIZER` ไว้ดูคำ) ; SBERT 384 มิติ `normalize_embeddings=True`
- ผลลัพธ์ทุกตัวเป็น `float32` ขนาด 12 × d เรียงตาม `items`

Part 3: setup, cache_vec, TF-IDF และ SBERT (2/2)

```
    X = np.asarray(builder(), dtype=np.float32)
    np.save(p, X); print("saved:", name, X.shape); return X

# ---------- Text: TF-IDF ----------
def build_tfidf():
    v = TfidfVectorizer(lowercase=True, ngram_range=(1, 2), min_df=1)
    return v.fit_transform(items["text_content"]).toarray(), v
X_text_tfidf, TFIDF_VECTORIZER = build_tfidf()

# ---------- Text Transformer: SBERT ----------
def _sbert():
    m = SentenceTransformer("sentence-transformers/all-MiniLM-L6-v2",
            device=str(DEVICE))
    return m.encode(items["text_content"].tolist(),
            show_progress_bar=True, normalize_embeddings=True)
X_text_sbert = cache_vec("text_sbert", _sbert)
```

อธิบายโค้ด

โครงที่ทุก encoder ใช้ร่วมกัน

- `cache_vec` รับ builder (ฟังก์ชันไม่มี argument) ⟶ ถ้ามีไฟล์ `.npy` แล้วโหลดทันที ไม่งั้นคำนวณแล้วบันทึกลง Drive ; `rebuild=True` บังคับคำนวณใหม่
- `DEVICE` เลือก GPU อัตโนมัติ ; โมเดล pretrained ทุกตัว `.eval()` + `torch.no_grad()` = ใช้เป็น feature extractor ไม่ฝึกต่อ
- TF-IDF ไม่ cache (เร็วและต้องเก็บ `TFIDF_VECTORIZER` ไว้ดูคำ) ; SBERT 384 มิติ `normalize_embeddings=True`
- ผลลัพธ์ทุกตัวเป็น `float32` ขนาด 12 × d เรียงตาม `items`

Part 3: TextCNN — สาธิตกลไก CNN กับข้อความ (1/2)

```
def build_textcnn_vectors(epochs=30, emb_dim=48, filters=32):
    texts = [str(x).lower().split() for x in items["text_content"]]
    vocab = {"<PAD>": 0, "<UNK>": 1}
    for t in texts:
        for w in t:
            if w not in vocab: vocab[w] = len(vocab)
    maxlen = max(4, min(80, max(len(t) for t in texts)))
    seq = np.zeros((len(texts), maxlen), dtype=np.int64)
    for i, t in enumerate(texts):
        ids = [vocab.get(w, 1) for w in t][:maxlen]
        seq[i, :len(ids)] = ids
    labels = items["genre"].str.split("[,|]",
            regex=True).str[0].fillna("Unknown")   # primary genre
    cats = {g: i for i, g in enumerate(sorted(labels.unique()))}
    y = np.array([cats[g] for g in labels], dtype=np.int64)

    class TextCNN(nn.Module):
        def __init__(self):
            super().__init__()
            self.emb = nn.Embedding(len(vocab), emb_dim, padding_idx=0)
            self.convs = nn.ModuleList([nn.Conv1d(emb_dim, filters, k)
                    for k in [2, 3, 4]])
            self.fc = nn.Linear(filters * 3, max(1, len(cats)))
```

อธิบายโค้ด

token ⟶ embedding ⟶ conv ⟶ max pooling

- **เตรียมข้อมูล**: สร้าง vocab เอง, แปลงเป็นลำดับ id ยาว `maxlen` (pad ด้วย 0) ; label = แนวหนังตัวแรก (primary genre) เพื่อให้มีสัญญาณฝึก
- **สถาปัตยกรรม**: Embedding 48 มิติ ⟶ Conv1d 3 ขนาดหน้าต่าง (2, 3, 4 คำ = n-gram detector) × 32 filter ⟶ ReLU ⟶ max-over-time pooling ⟶ ต่อกันเป็น 96 มิติ ⟶ Linear จำแนก genre
- ฝึก 30 epoch แบบ full-batch ด้วย Adam บน 12 ตัวอย่าง — **เพื่อสาธิตกลไกเท่านั้น** (overfit แน่นอน)
- ใช้ `model.features(...)` (ก่อน `fc`) เป็นเวกเตอร์หนัง แล้ว L2 ⟶ เทียบกับ TF-IDF/SBERT ได้ใน Part 3A
- ถ้ามี genre เดียว (`len(cats) == 1`) ข้ามการฝึก ใช้ค่าสุ่มเริ่มต้น

Part 3: TextCNN — สาธิตกลไก CNN กับข้อความ (2/2)

```
        def features(self, x):
            e = self.emb(x).transpose(1, 2)
                    # (B, emb, L)
            fs = [torch.relu(c(e)).max(dim=2).values for c in
                    self.convs]     # max-over-time pooling
            return torch.cat(fs, dim=1)
                    # (B, 96)
        def forward(self, x): return self.fc(self.features(x))

    model = TextCNN().to(DEVICE)
    if len(cats) > 1:
        opt = torch.optim.Adam(model.parameters(), lr=0.01)
        X = torch.tensor(seq, dtype=torch.long, device=DEVICE); Y =
                torch.tensor(y, dtype=torch.long, device=DEVICE)
        for _ in range(epochs):
            opt.zero_grad(); loss = nn.CrossEntropyLoss()(model(X), Y);
                    loss.backward(); opt.step()
    model.eval()
    with torch.no_grad():
        z = model.features(torch.tensor(seq, dtype=torch.long,
                device=DEVICE)).cpu().numpy()
    return normalize(z)
X_text_cnn = cache_vec("textcnn_demo", build_textcnn_vectors)
```

อธิบายโค้ด

token ⟶ embedding ⟶ conv ⟶ max pooling

- **เตรียมข้อมูล**: สร้าง vocab เอง, แปลงเป็นลำดับ id ยาว `maxlen` (pad ด้วย 0) ; label = แนวหนังตัวแรก (primary genre) เพื่อให้มีสัญญาณฝึก
- **สถาปัตยกรรม**: Embedding 48 มิติ ⟶ Conv1d 3 ขนาดหน้าต่าง (2, 3, 4 คำ = n-gram detector) × 32 filter ⟶ ReLU ⟶ max-over-time pooling ⟶ ต่อกันเป็น 96 มิติ ⟶ Linear จำแนก genre
- ฝึก 30 epoch แบบ full-batch ด้วย Adam บน 12 ตัวอย่าง — **เพื่อสาธิตกลไกเท่านั้น** (overfit แน่นอน)
- ใช้ `model.features(...)` (ก่อน `fc`) เป็นเวกเตอร์หนัง แล้ว L2 ⟶ เทียบกับ TF-IDF/SBERT ได้ใน Part 3A
- ถ้ามี genre เดียว (`len(cats) == 1`) ข้ามการฝึก ใช้ค่าสุ่มเริ่มต้น

Part 3: ResNet18 / ViT ที่ใช้ร่วมกันสำหรับภาพและวิดีโอ (1/2)

```
RW = ResNet18_Weights.DEFAULT
RESNET_PRE = RW.transforms()                          #
        resize/crop/normalize ตามที่โมเดลถูกฝึก
RESNET = resnet18(weights=RW); RESNET.fc = nn.Identity(); RESNET =
        RESNET.eval().to(DEVICE)

VW = ViT_B_16_Weights.DEFAULT
VIT_PRE = VW.transforms()
VIT = vit_b_16(weights=VW); VIT.heads = nn.Identity(); VIT =
        VIT.eval().to(DEVICE)

def encode_pil(images, model, prep, batch=8):
    rows = []
    for i in range(0, len(images), batch):
        x = torch.stack([prep(im.convert("RGB")) for im in images[i:i +
```

อธิบายโค้ด

ตัด head ออก = ได้ feature extractor

- ResNet18 (CNN): แทนที่ `fc` ด้วย `Identity` ⟶ output คือ global-average-pooled feature 512 มิติ ; ViT-B/16 (Transformer): แทนที่ `heads` ⟶ CLS token 768 มิติ
- `RW.transforms()` / `VW.transforms()` = preprocessing ที่ถูกต้องของแต่ละโมเดล (resize 224, normalize ด้วย mean/std ของ ImageNet) — ต้องใช้คู่กับ weight นั้น
- `encode_pil` รับ list ของ `PIL.Image` ทำ batch ละ 8 ⟶ ใช้ซ้ำได้ทั้งภาพโปสเตอร์, spectrogram ของเสียง และเฟรมวิดีโอ
- ภาพที่ไม่มีไฟล์ (`path is None`) แทนด้วยภาพดำ 224×224 เพื่อให้เมทริกซ์ครบ 12 แถว
- ทั้งสองผ่าน `normalize` (L2) ก่อนเก็บ cache

Part 3: ResNet18 / ViT ที่ใช้ร่วมกันสำหรับภาพและวิดีโอ (2/2)

```
                batch]]).to(DEVICE)
        with torch.no_grad(): rows.append(model(x).cpu().numpy())
    return np.vstack(rows)

# ---------- Image CNN / Transformer ----------
def _image_cnn():
    ims = [Image.open(p) if p else Image.new("RGB", (224, 224)) for p
           in items["image_path"]]
    return normalize(encode_pil(ims, RESNET, RESNET_PRE))
def _image_vit():
    ims = [Image.open(p) if p else Image.new("RGB", (224, 224)) for p
           in items["image_path"]]
    return normalize(encode_pil(ims, VIT, VIT_PRE))
X_image_cnn = cache_vec("image_resnet18", _image_cnn)     # 12 x 512
X_image_vit = cache_vec("image_vit", _image_vit)          # 12 x 768
```

อธิบายโค้ด

ตัด head ออก = ได้ feature extractor

- ResNet18 (CNN): แทนที่ `fc` ด้วย `Identity` ⟶ output คือ global-average-pooled feature 512 มิติ ; ViT-B/16 (Transformer): แทนที่ `heads` ⟶ CLS token 768 มิติ
- `RW.transforms()` / `VW.transforms()` = preprocessing ที่ถูกต้องของแต่ละโมเดล (resize 224, normalize ด้วย mean/std ของ ImageNet) — ต้องใช้คู่กับ weight นั้น
- `encode_pil` รับ list ของ `PIL.Image` ทำ batch ละ 8 ⟶ ใช้ซ้ำได้ทั้งภาพโปสเตอร์, spectrogram ของเสียง และเฟรมวิดีโอ
- ภาพที่ไม่มีไฟล์ (`path is None`) แทนด้วยภาพดำ 224×224 เพื่อให้เมทริกซ์ครบ 12 แถว
- ทั้งสองผ่าน `normalize` (L2) ก่อนเก็บ cache

Part 3: Audio — Log-Mel ⟶ ResNet18 และ Wav2Vec2 (1/2)

```
# ---------- Audio CNN: Log-Mel spectrogram -> CNN ----------
def audio_to_image(path, sr=16000):
    if not path: return Image.new("RGB", (224, 224))
    y, _ = librosa.load(path, sr=sr, mono=True)
    mel = librosa.feature.melspectrogram(y=y, sr=sr, n_mels=128,
            n_fft=1024, hop_length=256)
    db = librosa.power_to_db(mel, ref=np.max)
    lo, hi = float(db.min()), float(db.max())
    x = (db - lo) / (hi - lo) if hi > lo else np.zeros_like(db)
    return Image.fromarray(np.uint8(np.clip(x * 255, 0,
            255))).convert("RGB")
def _audio_cnn():
    ims = [audio_to_image(p) for p in items["audio_path"]]
    return normalize(encode_pil(ims, RESNET, RESNET_PRE))
X_audio_cnn = cache_vec("audio_logmel_resnet18", _audio_cnn)          #
        12 x 512

# ---------- Audio Transformer: Wav2Vec2 ----------
```

อธิบายโค้ด

สองเส้นทางของเสียง

- **เส้นทาง CNN**: เสียง ⟶ Log-Mel spectrogram (128 mel bands, dB, min-max ⟶ 0–255) ⟶ **แปลงเป็นภาพ RGB** ⟶ ResNet18 ตัวเดียวกับภาพ ⟶ 512 มิติ — เห็นชัดว่า “เสียงถูกมองเป็นภาพ”
- **เส้นทาง Transformer**: waveform 16 kHz (ตัด 12 วินาที) ⟶ Wav2Vec2 (`base-960h`) ⟶ `last_hidden_state` ต่อ time step ⟶ **mean pooling ตามเวลา** ⟶ 768 มิติ
- ทั้งคู่ resample เป็น 16 kHz mono ด้วย `librosa` ; ไม่มีไฟล์ ⟶ ภาพดำ / เวกเตอร์ศูนย์
- Wav2Vec2 ถูกฝึกเพื่อรู้จำเสียงพูด — กับเสียงประกอบหนังจึงเป็น feature ทั่วไปมากกว่า semantic ของเพลง (ข้อจำกัดที่ควรอภิปราย)
- ทั้งคู่ L2 ก่อน cache

Part 3: Audio — Log-Mel ⟶ ResNet18 และ Wav2Vec2 (2/2)

```python
def _audio_w2v():
    proc = AutoProcessor.from_pretrained("facebook/wav2vec2-base-960h")
    mod = AutoModel.from_pretrained("facebook/wav2vec2-base-960h").eval(
            ).to(DEVICE)
    rows = []
    for p in items["audio_path"]:
        if not p:
            rows.append(np.zeros(768)); continue
        y, _ = librosa.load(p, sr=16000, mono=True); y = y[:16000 * 12]
                # 12 วินาทีแรก
        inp = proc(y, sampling_rate=16000, return_tensors="pt")
        with torch.no_grad():
            z = mod(inp["input_values"].to(DEVICE)).last_hidden_state.me
                    an(1).squeeze(0).cpu().numpy()
        rows.append(z)
    return normalize(np.vstack(rows))
X_audio_transformer = cache_vec("audio_wav2vec2", _audio_w2v)
        # 12 x 768
```

อธิบายโค้ด

สองเส้นทางของเสียง

- **เส้นทาง CNN**: เสียง ⟶ Log-Mel spectrogram (128 mel bands, dB, min-max ⟶ 0–255) ⟶ **แปลงเป็นภาพ RGB** ⟶ ResNet18 ตัวเดียวกับภาพ ⟶ 512 มิติ — เห็นชัดว่า “เสียงถูกมองเป็นภาพ”
- **เส้นทาง Transformer**: waveform 16 kHz (ตัด 12 วินาที) ⟶ Wav2Vec2 (`base-960h`) ⟶ `last_hidden_state` ต่อ time step ⟶ **mean pooling ตามเวลา** ⟶ 768 มิติ
- ทั้งคู่ resample เป็น 16 kHz mono ด้วย `librosa` ; ไม่มีไฟล์ ⟶ ภาพดำ / เวกเตอร์ศูนย์
- Wav2Vec2 ถูกฝึกเพื่อรู้จำเสียงพูด — กับเสียงประกอบหนังจึงเป็น feature ทั่วไปมากกว่า semantic ของเพลง (ข้อจำกัดที่ควรอภิปราย)
- ทั้งคู่ L2 ก่อน cache

Part 3: Video — frame sampling ⟶ CNN/ViT ⟶ temporal mean pooling (1/2)

```python
def frames(path, n=8):
    if not path: return []
    cap = cv2.VideoCapture(str(path)); total =
            int(cap.get(cv2.CAP_PROP_FRAME_COUNT) or 0)
    if total <= 0: cap.release(); return []
    ids = np.linspace(0, total - 1, min(n, total)).astype(int); out = []
    for j in ids:
        cap.set(cv2.CAP_PROP_POS_FRAMES, int(j)); ok, f = cap.read()
        if ok: out.append(Image.fromarray(cv2.cvtColor(f,
                cv2.COLOR_BGR2RGB)))
    cap.release(); return out

def _video(model, prep, dim):
    rows = []
    for p in items["video_path"]:
        fs = frames(p)
```

อธิบายโค้ด

วิดีโอ = ลำดับภาพ

- `frames`: สุ่ม 8 เฟรมกระจายทั่วคลิป (`linspace`) แปลง BGR⟶RGB เป็น `PIL.Image`
- `_video`: encode ทุกเฟรมด้วย encoder **ภาพตัวเดิม** (ResNet หรือ ViT) แล้ว **mean pooling ตามเวลา** ⟶ 1 เวกเตอร์ต่อคลิป ; ไม่มีไฟล์ ⟶ เวกเตอร์ศูนย์ขนาด `dim`
- `lambda: _video(...)` ทำให้ส่งเป็น builder ให้ `cache_vec` ได้
- `VECTOR_LIBRARY` = 9 เวกเตอร์ (3 text + 2 image + 2 audio + 2 video) ทุกตัว 12 แถว ; TF-IDF ยังไม่ L2 (Part 3A จะ `transform_vectors(..., "L2")`)
- แนวคิดร่วม: encoder ⟶ pooling ⟶ L2 ทำให้ทุกสื่ออยู่ใน “รูปแบบเดียวกัน” สำหรับ Core

Part 3: Video — frame sampling ⟶ CNN/ViT ⟶ temporal mean pooling (2/2)

```
        rows.append(encode_pil(fs, model, prep).mean(0) if fs else
                np.zeros(dim))
    return normalize(np.vstack(rows))
X_video_cnn = cache_vec("video_resnet_mean", lambda: _video(RESNET,
        RESNET_PRE, 512))
X_video_vit = cache_vec("video_vit_mean", lambda: _video(VIT, VIT_PRE,
        768))

VECTOR_LIBRARY = {
    "Text-TFIDF": X_text_tfidf, "Text-CNN": X_text_cnn, "Text-SBERT":
            X_text_sbert,
    "Image-CNN": X_image_cnn, "Image-ViT": X_image_vit,
    "Audio-CNN": X_audio_cnn, "Audio-Wav2Vec2": X_audio_transformer,
    "Video-CNN": X_video_cnn, "Video-ViT": X_video_vit,
}
print({k: v.shape for k, v in VECTOR_LIBRARY.items()})
```

อธิบายโค้ด

วิดีโอ = ลำดับภาพ

- `frames`: สุ่ม 8 เฟรมกระจายทั่วคลิป (`linspace`) แปลง BGR⟶RGB เป็น `PIL.Image`
- `_video`: encode ทุกเฟรมด้วย encoder **ภาพตัวเดิม** (ResNet หรือ ViT) แล้ว **mean pooling ตามเวลา** ⟶ 1 เวกเตอร์ต่อคลิป ; ไม่มีไฟล์ ⟶ เวกเตอร์ศูนย์ขนาด `dim`
- `lambda: _video(...)` ทำให้ส่งเป็น builder ให้ `cache_vec` ได้
- `VECTOR_LIBRARY` = 9 เวกเตอร์ (3 text + 2 image + 2 audio + 2 video) ทุกตัว 12 แถว ; TF-IDF ยังไม่ L2 (Part 3A จะ `transform_vectors(..., "L2")`)
- แนวคิดร่วม: encoder ⟶ pooling ⟶ L2 ทำให้ทุกสื่ออยู่ใน “รูปแบบเดียวกัน” สำหรับ Core

สรุป 9 เวกเตอร์ใน VECTOR_LIBRARY

ชื่อ	Encoder	Pooling	มิติ	ประเภท
Text-TFIDF	TfidfVectorizer (1–2 gram)	—	|vocab|	lexical (sparse)
Text-CNN	TextCNN ฝึกเองจาก genre	max-over-time	96	CNN (demo)
Text-SBERT	all-MiniLM-L6-v2	sentence embedding	384	Transformer
Image-CNN	ResNet18 (ImageNet)	global average (ในโมเดล)	512	CNN
Image-ViT	ViT-B/16 (ImageNet)	CLS token	768	Transformer
Audio-CNN	Log-Mel ⟶ ResNet18	global average	512	CNN
Audio-Wav2Vec2	wav2vec2-base-960h	mean over time	768	Transformer
Video-CNN	8 frames ⟶ ResNet18	mean over frames	512	CNN
Video-ViT	8 frames ⟶ ViT-B/16	mean over frames	768	Transformer

ทุกตัว L2-normalize ⟶ cosine เทียบได้ภายใน space เดียวกัน

4

ประเมินรายสื่อและ Vector Transformation

Part 3A–3D และ Part 4

- `quick_eval` กับทุกเวกเตอร์: Text (3), Image (2), Audio (2), Video (2)
- Transformation 4 แบบบน Text-SBERT

Part 3A–3D: quick_eval รายสื่อ

```
# Part 3A — Text: TF-IDF vs TextCNN vs SBERT
for n in ["Text-TFIDF", "Text-CNN", "Text-SBERT"]:
    print("\n==", n, "==")
    display(quick_eval(transform_vectors(VECTOR_LIBRARY[n], "L2"),
            n).round(4))

# Part 3B — Image: CNN vs ViT
for n in ["Image-CNN", "Image-ViT"]:
    print("\n==", n, "=="); display(quick_eval(VECTOR_LIBRARY[n],
            n).round(4))

# Part 3C — Audio: Log-Mel CNN vs Wav2Vec2
for n in ["Audio-CNN", "Audio-Wav2Vec2"]:
    print("\n==", n, "=="); display(quick_eval(VECTOR_LIBRARY[n],
            n).round(4))

# Part 3D — Video: CNN vs ViT per frame + temporal pooling
for n in ["Video-CNN", "Video-ViT"]:
    print("\n==", n, "=="); display(quick_eval(VECTOR_LIBRARY[n],
            n).round(4))
```

อธิบายโค้ด

คำถามที่แต่ละส่วนตอบ

- 3A: ข้อความแบบ lexical (TF-IDF) vs CNN ฝึกเอง vs Transformer pretrained — SBERT มักดีที่สุด, TextCNN ต่ำเพราะฝึกจาก 12 ตัวอย่าง
- 3B / 3D: CNN (ResNet18) vs Transformer (ViT) กับภาพเดียวและกับเฟรมวิดีโอเฉลี่ย — ใช้ encoder เดียวกันจึงเทียบ “ภาพนิ่ง vs ภาพเคลื่อนไหว” ได้
- 3C: เสียงผ่านภาพ (Log-Mel + CNN) vs ผ่านโมเดลเสียงโดยตรง (Wav2Vec2)
- ทุกตารางมี 3 แถว (profile) × HR@3 / MRR / nDCG@3 / Coverage / Gini / ILD / Novelty / Calibration — protocol เดียวกัน (LOO, Full Catalog, K=3)
- **ระวัง**: ILD วัดใน space ของสื่อนั้น ; ค่าสูงของ TF-IDF ไม่ได้แปลว่าหลากหลายกว่า ViT

Part 4: Transformation 4 แบบบน Text-SBERT

```
rows = []
for mode in ["L2", "Center+L2", "Standard+L2", "PCA+L2"]:
    X = transform_vectors(X_text_sbert, mode)
    s, _ = evaluate_vector_space(X, f"SBERT/{mode}", "Rating-centered")
    rows.append({"Transformation": mode, **s.to_dict()})
display(pd.DataFrame(rows).round(4))
```

อธิบายโค้ด

geometry เปลี่ยน ⟶ profile และ ranking เปลี่ยน

- ตรึง representation (SBERT) และ profile (Rating-centered) ⟶ ต่างเฉพาะ transformation
- Center+L2 มักช่วย embedding จาก Transformer เพราะลบทิศทางร่วม (anisotropy) ทำให้ cosine กระจายกว้างขึ้น
- PCA+L2 เหลือ ≤ 11 มิติ (n−1) จาก 384 ⟶ ตัด noise แต่ก็อาจตัดสัญญาณ ; ผลขึ้นกับว่า 12 เรื่องแปรผันในกี่ทิศทาง
- `label=f"SBERT/{mode}"` ถูกใส่ในคอลัมน์ `Label` ของ summary
- อ่าน HR/nDCG คู่กับ Coverage/ILD — transformation ที่ทำให้ cosine กระจายมากขึ้นมักเพิ่ม ILD โดยอัตโนมัติ

5

Early Fusion และ Late Fusion

Part 5 – Part 6: รวม 4 สื่อสองวิธี แล้วเทียบ 3 profile บน fused space

- `early_fusion(vector_dict, weights)` ต่อบล็อกถ่วงน้ำหนัก √w
- `late_fusion_rank` / `evaluate_late_fusion` รวมคะแนน cosine รายสื่อ

Part 5: early_fusion และ DEFAULT_MODALITIES

```python
def early_fusion(vector_dict, weights=None):
    names = list(vector_dict)
    weights = weights or {n: 1.0 for n in names}
    blocks = []
    for n in names:
        X = normalize(vector_dict[n])                          # L2
                แต่ละบล็อกก่อน
        blocks.append(np.sqrt(float(weights.get(n, 1.0))) * X)
    return normalize(np.hstack(blocks))                        # L2
            เวกเตอร์รวมอีกครั้ง

DEFAULT_MODALITIES = {
    "text":  X_text_sbert,          # Transformer
    "image": X_image_cnn,           # CNN
    "audio": X_audio_transformer,   # Transformer
    "video": X_video_vit,           # Transformer
}
X_fused = early_fusion(DEFAULT_MODALITIES)                      # 12 x
        (384+512+768+768)
```

อธิบายโค้ด

ทำไมต้อง √w และ L2 สองชั้น

- L2 แต่ละบล็อกก่อน ⟶ ทุกสื่อมี “ความยาว 1” เท่ากัน ไม่ว่ามิติจะเป็น 384 หรือ 768 ⟶ **ไม่มีสื่อใดครอบงำเพราะสเกล**
- คูณด้วย √w: dot product ของเวกเตอร์รวม = $\sum$ w_m · cos_m ⟶ น้ำหนัก w มีผลเชิงเส้นต่อคะแนน (ถ้าคูณ w ตรง ๆ จะได้ w^2)
- L2 รอบนอกทำให้ cosine ของเวกเตอร์รวมอยู่ในช่วงปกติ
- `DEFAULT_MODALITIES` = hybrid: SBERT + ResNet + Wav2Vec2 + ViT ; เปลี่ยน dict นี้ = เปลี่ยนชุด encoder (LAB 2 ทำ CNN-only / Transformer-only)
- ผลลัพธ์ `X_fused` ใช้กับ Core ได้เหมือนเวกเตอร์เดี่ยว

Part 5: late_fusion_rank และ evaluate_late_fusion (1/2)

```
def late_fusion_rank(uid, vector_dict, rdf,
        profile_method="Rating-centered",
                    weights=None, candidates=None, alpha=1, beta=1):
    weights = weights or {n: 1 / len(vector_dict) for n in vector_dict}
    seen = set(rdf.loc[rdf["user_id"].astype(str) == str(uid),
            "item_id"].astype(str))
    cand = candidates if candidates is not None else [i for i in
            items["item_id"] if i not in seen]
    total = {iid: 0.0 for iid in cand}
    for name, X in vector_dict.items():
        ranked, s, _ = rank_by_vector(uid, X, rdf, profile_method,
                cand, alpha, beta)
        for iid in cand: total[iid] += weights.get(name, 0) *
                float(s[ITEM_INDEX[iid]])
    ranked = sorted(cand, key=lambda x: total[x], reverse=True)
    return ranked, total

def evaluate_late_fusion(vector_dict, label="Late Fusion",
        profile_method="Rating-centered",
                        k=3, folds=None, weights=None,
                                candidate_mode="Full Catalog",
                                        n_neg=20, seed=42):
    folds = folds or split_loo(ratings); rng =
            np.random.default_rng(seed)
    rows = []; all_recs = []
```

อธิบายโค้ด

Late = โปรไฟล์ต่อสื่อ แล้วรวมคะแนน

- `late_fusion_rank` เรียก `rank_by_vector` **แยกต่อสื่อ** (โปรไฟล์ text, โปรไฟล์ image, ...) แล้วรวม $\Sigma\ w_m \cdot cos_m$ ต่อ candidate ; น้ำหนักเริ่มต้นเท่ากัน (1/4)
- `evaluate_late_fusion` = engine เดิมที่แทน `rank_by_vector` ด้วย `late_fusion_rank` ; ILD **วัดใน** `Xdiv = early_fusion(...)` เพราะไม่มี space เดียวให้วัด
- ทั้ง Early และ Late ใช้ encoder ชุดเดียวกัน (`DEFAULT_MODALITIES`) ⟶ ต่างกันเฉพาะ **จุดที่รวม** (ก่อน/หลัง cosine)
- Early: โปรไฟล์เดียวใน space ต่อกัน ; Late: อธิบายได้ว่าสื่อใดดันเรื่องนี้ขึ้น (ดู contribution ได้) และปรับน้ำหนักโดยไม่ต้องสร้างเวกเตอร์ใหม่
- ผลบน 12 เรื่องมักใกล้กัน — ใช้เพื่อเห็นโครงสร้างของสองวิธี

Part 5: late_fusion_rank และ evaluate_late_fusion (2/2)

```
    Xdiv = early_fusion(vector_dict)          # ILD ใช้ early-fused
           space เพื่อวัดความหลากหลายร่วม
    for uid, train, heldout in folds:
        ...  # เตรียม candidate เหมือน evaluate_vector_space
        ranked, sc = late_fusion_rank(uid, vector_dict, train,
               profile_method, weights, cand)
        top = ranked[:k]; all_recs += top
        y = np.array([1 if i == heldout else 0 for i in ranked]); ys =
               np.array([sc[i] for i in ranked])
        rows.append({... metric เดิม ..., "AUC": roc_auc_score(y, ys)
               if len(np.unique(y)) == 2 else np.nan,
                    "ILD": ild(top, Xdiv), "Novelty": novelty(top),
                           "Calibration": calibration_genre(uid, top,
                                  train)})
    d = pd.DataFrame(rows); s = d.mean(numeric_only=True).to_dict()
    s["Coverage"] = coverage(all_recs); s["Gini"] = gini(all_recs);
           s["Label"] = label
    return pd.Series(s), d

s1, _ = evaluate_vector_space(X_fused, "Early Fusion",
       "Rating-centered")
s2, _ = evaluate_late_fusion(DEFAULT_MODALITIES, "Late Fusion",
       "Rating-centered")
display(pd.DataFrame([s1, s2]).round(4))
```

อธิบายโค้ด

Late = โปรไฟล์ต่อสื่อ แล้วรวมคะแนน

- `late_fusion_rank` เรียก `rank_by_vector` **แยกต่อสื่อ** (โปรไฟล์ text, โปรไฟล์ image, ...) แล้วรวม $\Sigma\ w_m \cdot cos_m$ ต่อ candidate ; น้ำหนักเริ่มต้นเท่ากัน (1/4)
- `evaluate_late_fusion` = engine เดิมที่แทน `rank_by_vector` ด้วย `late_fusion_rank` ; ILD **วัดใน** `Xdiv = early_fusion(...)` เพราะไม่มี space เดียวให้วัด
- ทั้ง Early และ Late ใช้ encoder ชุดเดียวกัน (`DEFAULT_MODALITIES`) ⟶ ต่างกันเฉพาะ **จุดที่รวม** (ก่อน/หลัง cosine)
- Early: โปรไฟล์เดียวใน space ต่อกัน ; Late: อธิบายได้ว่าสื่อใดดันเรื่องนี้ขึ้น (ดู contribution ได้) และปรับน้ำหนักโดยไม่ต้องสร้างเวกเตอร์ใหม่
- ผลบน 12 เรื่องมักใกล้กัน — ใช้เพื่อเห็นโครงสร้างของสองวิธี

Part 6: 3 User Profile บน Multimodal Fusion

```
rows = []
for p in PROFILE_METHODS:
    s, _ = evaluate_vector_space(X_fused, "Multimodal", p)
    rows.append({"Profile": p, **s.to_dict()})
display(pd.DataFrame(rows).round(4))
```

อธิบายโค้ด

profile บน space ที่รวม 4 สื่อ

- ตรึง `X_fused` (hybrid) ⟶ ต่างเฉพาะสูตรโปรไฟล์ Positive-only / Rating-centered / Rocchio
- ใน space ที่มี 2,400+ มิติจากหลายสื่อ negative feedback (Rating-centered, Rocchio) อาจเปลี่ยนอันดับมากกว่าใน space ข้อความเดียว — สังเกตจาก Coverage/Gini
- เทียบกับตาราง Part 3A–3D จะเห็นว่า fusion ช่วยหรือไม่เมื่อเทียบกับสื่อที่ดีที่สุดเพียงสื่อเดียว (มัก SBERT)
- ตารางนี้คือรูปแบบเดียวกับ `quick_eval` แต่เขียน loop ตรง ๆ

6

GUI: History + Profile + Top-K + Media

Part 7: gr.Blocks ที่แสดงภาพ เสียง วิดีโอของอันดับ 1

- `FUSED_SPACES` = 3 transformation ของ `X_fused`
- `gui` คืน 6 output: history, profile preview, recommendations, image, audio, video

Part 7: gui และ gr.Blocks (1/2)

```
FUSED_SPACES = {"Fused/L2": transform_vectors(X_fused, "L2"),
                "Fused/Center+L2": transform_vectors(X_fused,
                        "Center+L2"),
                "Fused/Standard+L2": transform_vectors(X_fused,
                        "Standard+L2")}

def gui(uid, space, profile, alpha, beta, k):
    X = FUSED_SPACES[space]
    ranked, s, p = rank_by_vector(uid, X, ratings, profile,
            alpha=float(alpha), beta=float(beta))
    top = ranked[:int(k)]
    hist = ratings[ratings["user_id"].astype(str) == str(uid)].merge(
        items[["item_id", "title", "genre"]], on="item_id", how="left"
    )[["title", "genre", "rating"]].sort_values("rating",
            ascending=False)
    rec = []
    for rnk, iid in enumerate(top, 1):
        row = items.iloc[ITEM_INDEX[iid]]
        rec.append({"Rank": rnk, "Title": row["title"], "Genre":
                row["genre"],
                    "Score": round(float(s[ITEM_INDEX[iid]]), 4),
                            "Synopsis": row["synopsis"]})
    first = items.iloc[ITEM_INDEX[top[0]]] if top else None
    profile_preview = pd.DataFrame({"dim": np.arange(min(20, len(p))),
            "value": p[:20]})
    return (hist, profile_preview, pd.DataFrame(rec),
            first["image_path"] if first is not None else None,
```

อธิบายโค้ด

เห็นสื่อจริงของสิ่งที่ระบบแนะนำ

- `gr.Blocks` แทน `gr.Interface`: จัด layout เป็นแถวและผูก event เอง (`run.click(fn, inputs, outputs)`)
- `FUSED_SPACES` คำนวณล่วงหน้า 3 space ⟶ เลือกได้โดยไม่คำนวณซ้ำ
- Profile preview = 20 มิติแรกของ `p` (fused space ไม่มีชื่อมิติ จึงแสดงเป็นตัวเลข)
- **สื่อของอันดับ 1**: คืน path ของ image/audio/video ให้ `gr.Image/Audio/Video` แสดงตรง ๆ ; `None` เมื่อไม่มีไฟล์ ⟶ ช่องว่าง
- แนวทางสอน: ดู History ⟶ ดู Top-K + โปสเตอร์/เสียง/คลิปของอันดับ 1 ⟶ สลับ profile/transformation ⟶ ถามว่าเรื่องที่แนะนำคล้ายประวัติในสื่อใด

Part 7: gui และ gr.Blocks (2/2)

```
            first["audio_path"] if first is not None else None,
            first["video_path"] if first is not None else None)

with gr.Blocks() as demo:
    gr.Markdown("# Multimodal Recommender Demo")
    with gr.Row():
        uid = gr.Dropdown(sorted(ratings["user_id"].unique()),
                label="User")
        space = gr.Dropdown(list(FUSED_SPACES), value="Fused/L2",
                label="Transformation")
        prof = gr.Dropdown(PROFILE_METHODS, value="Rating-centered",
                label="User Profile")
    with gr.Row():
        a = gr.Slider(0, 2, 1, 0.1, label="Rocchio alpha"); b =
                gr.Slider(0, 2, 1, 0.1, label="Rocchio beta")
        k = gr.Slider(1, 5, 3, 1, label="Top-K")
    run = gr.Button("Recommend")
    hist = gr.Dataframe(label="User History"); pv =
            gr.Dataframe(label="User Profile Vector (preview)")
    rec = gr.Dataframe(label="Recommended Movies")
    with gr.Row():
        im = gr.Image(label="Top-1 Image"); au = gr.Audio(label="Top-1
                Audio"); vi = gr.Video(label="Top-1 Video")
    run.click(gui, [uid, space, prof, a, b, k], [hist, pv, rec, im, au,
            vi])
demo.launch(debug=False)
```

อธิบายโค้ด

เห็นสื่อจริงของสิ่งที่ระบบแนะนำ

- `gr.Blocks` แทน `gr.Interface`: จัด layout เป็นแถวและผูก event เอง (`run.click(fn, inputs, outputs)`)
- `FUSED_SPACES` คำนวณล่วงหน้า 3 space ⟶ เลือกได้โดยไม่คำนวณซ้ำ
- Profile preview = 20 มิติแรกของ `p` (fused space ไม่มีชื่อมิติ จึงแสดงเป็นตัวเลข)
- **สื่อของอันดับ 1**: คืน path ของ image/audio/video ให้ `gr.Image/Audio/Video` แสดงตรง ๆ ; `None` เมื่อไม่มีไฟล์ ⟶ ช่องว่าง
- แนวทางสอน: ดู History ⟶ ดู Top-K + โปสเตอร์/เสียง/คลิปของอันดับ 1 ⟶ สลับ profile/transformation ⟶ ถามว่าเรื่องที่แนะนำคล้ายประวัติในสื่อใด

สรุปแนวคิดของ Source Code Multimodal LAB 1

Take-away

- **Adapter / Core / Encoders แยกชั้นชัดเจน:** Part 1 ขึ้นกับ Dataset, Part 2 ขึ้นกับ schema เท่านั้น, Part 3 ขึ้นกับสื่อ ⟶ เปลี่ยนชั้นใดชั้นหนึ่งได้อิสระ
- **ทุกสื่อจบที่รูปแบบเดียวกัน:** encoder (CNN/Transformer) ⟶ pooling (global average / CLS / max-over-time / mean over time-frames) ⟶ L2 ⟶ เมทริกซ์ 12 × d ใน `VECTOR_LIBRARY` + cache `.npy`
- **เสียงและวิดีโอใช้ encoder ภาพซ้ำ:** Log-Mel เป็นภาพ ⟶ ResNet ; เฟรมวิดีโอ ⟶ ResNet/ViT ⟶ mean ; ทางเลือก Transformer เฉพาะสื่อ: Wav2Vec2
- **Core ไม่รู้จักสื่อ:** profile / rank / metric / split / `evaluate_vector_space` / `quick_eval` ทำงานกับ `X` ใด ๆ ⟶ ประเมิน 9 เวกเตอร์ + fusion ด้วยมาตรวัดเดียว
- **Fusion สองแบบ:** Early = √w-weighted concat แล้ว L2 (โปรไฟล์เดียว) ; Late = โปรไฟล์ต่อสื่อแล้วรวม $\sum$ w•cos (อธิบายได้, ILD วัดใน early space)
- **อ่านผลอย่างระวัง:** ILD ข้าม space เทียบไม่ได้, 12 เรื่องผลแกว่ง, TextCNN/Wav2Vec2 เป็นการสาธิตกลไก ; GUI แสดงสื่อจริงเพื่อตีความเชิงคุณภาพ

LAB 1 = สร้างและวัดเวกเตอร์หลายสื่อ • LAB 2 = experiment matrix • LAB 3 = predictive pipeline